Planificador curricular con predicción de riesgo

Trabajo Práctico Integrador — Lógica y Teoría de Grafos

De: Tomás Monzón y Morena Randisi

1. Descripción del problema y decisiones de diseño

1.1 El problema

La carrera de Abogacía (UNR, plan 2016) tiene 44 materias con correlatividades. El plan sugerido es igual para todos los estudiantes, pero el riesgo de no aprobar una materia depende del historial de cada uno: su promedio, las notas en las correlativas, cuántas veces cursó la materia y cuántas materias cursa a la vez. El sistema propuesto estima ese riesgo con un modelo de clasificación y lo usa para sugerir un orden de cursada que respete las correlatividades y priorice las materias de menor riesgo.

1.2 Por qué un grafo

El plan de estudios es un grafo dirigido acíclico (DAG): cada nodo es una materia y cada arista A → B indica que A debe estar aprobada para cursar B. Un grafo permite razonar sobre las dependencias (qué materias están habilitadas, qué materias desbloquea cada una) y aplicar un algoritmo de recorrido sobre costos.

1.3 Modelado

- **Nodos:** las 44 materias, más dos nodos auxiliares: “INICIO” (conectado a las materias sin correlativas) y “RECIBIDO” (al que llegan las materias sin sucesoras).
- **Aristas:** una por cada correlatividad. El peso de la arista A → B es el costo de cursar B (ver sección 5).
- **Regla de habilitación:** una materia está habilitada si no está aprobada ni en curso y todas sus correlativas están aprobadas.
- **Estados de una materia para el estudiante:** no cursada, aprobada, desaprobada. Una materia desaprobada puede volver a estar habilitada (recursada).

1.4 Supuestos

- Se aprueba con nota mayor o igual a 6. Las notas de materias aprobadas están entre 6 y 10.
- El plan se leyó de un archivo JSON(“plan_estudios.json”),lo usan tanto el generador de datos como el grafo.
- Se dejaron de lado las materias optativas.

- Decidimos que la clasificación sea de one vs rest: siendo la clase positiva aprobó contra reprobó como 0

2. Dataset

2.1 Tipo de dataset

Se usa un **dataset sintético**, generado con el script backend/data/generador.py. No se usaron datos reales porque los historiales académicos son privados y difíciles de conseguir. Como el generador define las reglas, **el modelo va a descubrir esas reglas**: es esperable y se declara explícitamente.

2.2 Procedimiento de generación

Se simulan 1500 estudiantes hasta 12 cuatrimestres. Cada estudiante tiene tres variables latentes que el modelo **no ve**, extraídas de una normal estándar: habilidad, constancia y carga_ext (carga externa, por ejemplo trabajo).

En cada cuatrimestre:

1. Se calculan las materias habilitadas (no aprobadas y con todas las correlativas aprobadas).
2. Se elige cuántas materias cursa (entre 1 y 4) según round(2.5 + 1.2·constancia − 0.8·carga_ext + ruido), acotado.
3. Se eligen al azar esas materias entre las habilitadas.
4. Para cada una se calcula la probabilidad de aprobar con una regresión logística oculta y se sortea el resultado.

La probabilidad de aprobar sigue este logit:

```
None
logit = 1.0
      + 0.5 · habilidad
      + 0.45 · (promedio_correlativas − 6.0) / 1.5
      + 0.25 · (peor_nota_correlativa − 5.5) / 2.0
      − 4.0 · (dificultad − 0.55)
      − 0.45 · (n_materias − 2.5)
      − 0.4 · veces_recursada
      + 0.3 · constancia
      − 0.2 · carga_ext
      + ruido  (normal, desvío 0.85)
```

Si el estudiante aprueba, su nota se genera como 6.5 + 1.8·habilidad − 2.0·(dificultad − 0.55) más ruido, acotada entre 6 y 10.

2.3 Dificultad por materia

La dificultad de cada materia (entre 0.30 y 0.85) es una **estimación subjetiva de los autores**, basada en el tipo de materia: las de derecho de fondo y las procesales (Penal I, Procesal I y III, Obligaciones, Contratos) son las más difíciles; las prácticas profesionales, el taller, el idioma y las de integración son las más blandas. **La dificultad solo la usa el generador; no es feature del modelo.** En un sistema real se obtendría de tasas históricas de aprobación.

2.4 Variables (features) del modelo

Todas se calculan antes de cursar la materia, para evitar fuga de información (*leakage*):

Variable	Descripción
promedio_general	Media de las notas de las materias aprobadas (6.0 si no aprobó ninguna)
promedio_correlativas	Media de las notas de las correlativas directas (6.0 si no tiene)
peor_nota_correlativa	Mínimo de esas notas (6.0 si no tiene correlativas)
veces_recursada	Intentos previos de esa materia (0 a 5)
n_materias_cuatrimestre	Cantidad de materias que cursa en el cuatrimestre (1 a 4)
avance_carrera	Fracción del plan aprobada
materia	Identificador de la materia, tratado como variable categórica

La variable objetivo es aprobo (1 si aprobó, 0 si no).

Decisión sobre los valores faltantes: cuando no hay datos (sin notas o sin correlativas) se usa 6.0, la misma regla en el generador y en el backend.

Variables descartadas: estudiante_id (el modelo memorizaría individuos), cuatrimestre (redundante con avance_carrera), y la nota final (fuga del objetivo). La tasa de aprobación histórica por materia se calcula en el generador pero **no se usa como feature**, porque

calcularla con todo el dataset filtraría información del conjunto de test; en su lugar se usa materia como categórica.

3. Metodología de entrenamiento del modelo

3.1 Modelo elegido

Regresión logística. El objetivo es obtener **probabilidades**, no sólo clases, y es un modelo interpretable: los coeficientes permiten verificar que el modelo aprendió relaciones con sentido. Se implementó con un Pipeline de scikit-learn: StandardScaler para las variables numéricas y OneHotEncoder para materia.

3.2 Partición de datos

Se separó el 20% para test con GroupShuffleSplit, **agrupando por estudiante**, de modo que ningún estudiante aparece a la vez en entrenamiento y en test. Una partición aleatoria por filas habría filtrado información, porque las filas de un mismo estudiante están correlacionadas. 1200 estudiantes para train y 300 estudiantes para test.

3.3 Selección de hiperparámetros

Se buscó el parámetro de regularización C entre 0.001, 0.01, 0.1, 1, 10 y 100 con GridSearchCV y validación cruzada GroupKFold de 5 particiones (también agrupada por estudiante), usando como métrica neg_log_loss. Se eligió log loss porque el sistema usa las probabilidades como pesos del grafo, y el log loss penaliza probabilidades mal calibradas.

Mejor C elegido: 1, mejor score: -0.5563755540357087

3.4 Métricas y resultados

En test se calculó:

- **Log loss: 0.5499392714216734**
- **Matriz de confusión, precisión, recall y F1** para umbrales 0.3 a 0.8:

  [[237 2602]

  [70 5727]]

  precision: 0.6875975507263777

  recall: 0.987924788683802

  f1:   0.8108452498938128

  [[533 2306]

  [233 5564]]

precision: 0.7069885641677255

recall: 0.9598067966189409

f1: 0.8142240433160167

[[1052 1787]

[642 5155]]

precision: 0.7425813886488044

recall: 0.8892530619285838

f1: 0.8093256927545334

[[1602 1237]

[1336 4461]]

precision: 0.782906282906283

recall: 0.7695359668794204

f1: 0.776163549369291

[[2121 718]

[2351 3446]]

precision: 0.8275696445725265

recall: 0.5944454027945489

f1: 0.6918984037747214

[[2549 290]

[3680 2117]]

precision: 0.8795180722891566

recall: 0.3651888908055891

f1: 0.5160897123354461

- **Curva de calibración** (probabilidad predicha media vs. tasa real por tramo de probabilidad):

tramo n prob_predicha tasa_real

(0.0, 0.1] 1 0.075 0.000

(0.1, 0.2]	**84**	**0.165**	**0.214**
(0.2, 0.3]	**222**	**0.257**	**0.234**
(0.3, 0.4]	**459**	**0.357**	**0.355**
(0.4, 0.5]	**928**	**0.456**	**0.441**
(0.5, 0.6]	**1244**	**0.552**	**0.558**
(0.6, 0.7]	**1534**	**0.651**	**0.662**
(0.7, 0.8]	**1757**	**0.750**	**0.756**
(0.8, 0.9]	**1588**	**0.847**	**0.849**
(0.9, 1.0]	**819**	**0.935**	**0.938**

Como el sistema usa la probabilidad y no la clase, el umbral de decisión no es relevante para el algoritmo de grafos.

3.4.1 Interpretación de coeficientes

Los coeficientes de las variables numéricas deberían tener el signo esperado (más recursadas y más materias simultáneas bajan la probabilidad; mejores notas la suben).

num__veces_recursada	**-0.337594**
num__n_materias_cuatrimestre	**-0.195254**
num__peor_nota_correlativa	**0.125369**
num__avance_carrera	**0.135240**
num__promedio_general	**0.290167**
num__promedio_correlativas	**0.371131**

3.5 Análisis de errores y hallazgo importante

La primera versión del modelo devolvía probabilidades casi uniformes (alrededor de 0.65 para todas las materias), e incluso una materia de primer año salía con menos probabilidad que una de quinto. Identificamos dos causas:

1. **Sesgo de supervivencia:** a las materias de años altos solo llegan los estudiantes que aprobaron todo lo anterior, es decir, los de mayor habilidad. Como esa variable es oculta para el modelo, este atribuía a la materia lo que era mérito de los estudiantes.

2. **Efecto de la dificultad demasiado débil** en el generador (coeficiente 1.6 y dificultades entre 0.35 y 0.70).

Los pesos del grafo son −log(P), así que con probabilidades casi iguales el algoritmo no tenía nada que optimizar. Se corrigió el generador (coeficiente de dificultad de 1.6 a 4.0, peso de la habilidad de 0.8 a 0.5, y dificultades ampliadas al rango 0.30–0.85), se regeneró el dataset y se reentrenó. Después de eso las tasas observadas de aprobación pasaron a depender de la dificultad (por ejemplo, Penal I ≈ 0.49 y Procesal I ≈ 0.53 contra Taller ≈ 0.82).

4. Algoritmo de grafos

4.1 Dijkstra adaptado a correlatividades

El enunciado pide Dijkstra para calcular el camino de menor riesgo acumulado respetando las correlatividades. El Dijkstra clásico encuentra un camino entre dos nodos, pero en un plan con correlativas múltiples, para cursar una materia hay que aprobar **todas** sus correlativas, no solo una rama. Por eso se adaptó el algoritmo (dijkstra_correlativas en backend/grafo.py):

1. costos[INICIO] = 0 y el resto en infinito. Los nodos pendientes son todos.
2. En cada iteración se consideran **candidatos** los nodos pendientes con costo finito y cuyas correlativas (predecesores, sin contar INICIO) ya fueron tomadas.
3. Se toma el candidato de menor costo, se lo agrega al orden y se relajan sus aristas salientes: costos[x] = min(costos[x], costos[v] + w(v, x)).
4. Se repite hasta que no quedan candidatos.

Orden de la salida: secuencia en que se toman los nodos. Como cada nodo se toma recién cuando todas sus correlativas ya fueron tomadas, **el orden respeta todas las correlatividades** y, entre las materias habilitadas, prefiere la de menor costo acumulado. El orden obtenido se verifica con una aserción en main.py.

4.2 Qué significa el resultado

El costo costos[x] es el riesgo acumulado por la cadena a través de la cual se alcanzó x; no es la suma de los costos de todos los requisitos. Por eso el resultado principal del sistema está en orden (la secuencia sugerida), y no un camino único entre INICIO y RECIBIDO. Como todas las materias deben aprobarse para recibirse, el camino mínimo clásico entre INICIO y RECIBIDO es trivial (pasa por una materia sin correlativas ni sucesoras, como Idioma Extranjero) y no informa nada útil.

5. Integración del modelo y el algoritmo de grafos

5.1 De la probabilidad al peso

Para cada materia pendiente, el backend arma el vector de features con el perfil actual del estudiante y obtiene P(aprobar) con predict_proba. El peso de cada arista A → B es:

```
None
peso(A → B) = −log( P(aprobar B) )
```

Esta transformación es razonable por tres motivos:

- Es siempre mayor o igual a 0 (Dijkstra lo requiere).
- Crece cuando la probabilidad baja: las materias riesgosas "cuestan" más.
- Sumar −log(P) a lo largo de una cadena equivale a minimizar el negativo del logaritmo del producto de probabilidades, es decir, maximizar la probabilidad de aprobar todas las materias de la cadena (bajo independencia).

Se acota P inferiormente en 1e-6 para evitar log(0). Las materias ya aprobadas y el nodo RECIBIDO reciben probabilidad 1, por lo que cuestan 0.

5.2 Materias aún no habilitadas

El generador solo creó filas de materias habilitadas, así que el modelo nunca vio una materia con correlativas sin aprobar. Para poder asignar pesos a todo el grafo, a las materias no habilitadas se les imputa un perfil proyectado:

- Las notas de correlativas no aprobadas se reemplazan por el promedio general actual.
- avance_carrera se calcula como (aprobadas ∪ ancestros de la materia) / total, es decir, el avance mínimo que tendrá el estudiante al llegar a ella.

Es una extrapolación razonable pero se declara como limitación (sección 8).

5.3 Reacción del algoritmo al modelo

Si cambia el perfil (otra nota, otra cantidad de recursadas, otro número de materias por cuatrimestre), cambian las probabilidades, por lo tanto los pesos, y puede cambiar la secuencia sugerida. En el modo simulación, la página muestra lado a lado las probabilidades y la secuencia antes y después del cambio hipotético.

6. Backend y frontend

6.1 Backend (FastAPI)

- GET /plan: devuelve las materias con año y correlativas.
- POST /sugerencia: recibe el historial {id_materia: {aprobada, nota, veces}} y n_materias; valida; arma el estudiante; predice las probabilidades; asigna los pesos; corre el Dijkstra adaptado y devuelve probabilidades, secuencia sugerida y materias aprobadas.

Validaciones: una materia con registro debe tener todas sus correlativas aprobadas; una aprobada requiere nota entre 6 y 10 y al menos una cursada (y como máximo entre 5

recursadas); n_materias se limita a 1–4 (el rango que vio el generador). El grafo base se copia en cada pedido para que los pesos de un pedido no pisen a otro.

6.2 Frontend (React + TypeScript + React Flow)

- Grafo de correlatividades con una columna por año y cada materia coloreada de rojo (riesgosa) a verde (segura), con su porcentaje de aprobar.
- Botón **"Ver camino sugerido"**: numera las materias en el orden sugerido, atenúa las que no están en la secuencia y muestra la lista en un panel lateral.
- **Carga de historial:** al hacer click en una materia habilitada se elige su estado (sin cursar, cursada sin aprobar, aprobada), la nota y la cantidad de veces cursada. Cada cambio recalcula todo en el backend.
- Las materias bloqueadas (con 🔒) no pueden cargarse; al quitar una materia aprobada se borra en cascada el historial de las que dependen de ella.
- **Modo simulación ("¿Y si...?"):** sobre una materia habilitada, simula que se aprueba (con una nota) o que no se aprueba, sin modificar el historial real, y muestra el cambio promedio de probabilidades, las materias que más cambian y las secuencias antes y después. Permite aplicar el escenario al historial o descartarlo.

7. Uso de inteligencia artificial

Herramientas utilizadas: Claude (Anthropic) como asistente conversacional y DeepSeek.

Para qué se usó:

- Entender el enunciado y el flujo del sistema.
- Definir la estructura del repositorio y del .gitignore.
- Diseñar la clase Estudiante, las reglas de habilitación y la función de pesos.
- Revisar código y explicar errores (imports, rutas, nombres de variables).
- Proponer el esqueleto de la API y del frontend.
- Analizar por qué las probabilidades del modelo eran casi uniformes.

Casos en que la IA se equivocó o propuso algo incorrecto:

- Al revisar habilitadas, la IA anticipó que con la materia 5 aprobada sólo aparecería la 7; en realidad también aparece la 12, porque ambas tienen a la 5 como única correlativa. Se detectó al ejecutar y comparar con el plan.
- En el borrador de manejoPredicciones.py se calculaba previas pero luego avance_carrera usaba otra expresión. Se detectó al releer el código.
- El primer diagnóstico de un error de importación (Path) no coincidía con el código real porque el repositorio subido estaba desactualizado respecto del local. Se resolvió verificando el código local.

8. Limitaciones conocidas y posibles extensiones

Limitaciones

- **Datos sintéticos:** el modelo aprende las reglas del generador, no el comportamiento de estudiantes reales. Las dificultades por materia son estimaciones subjetivas.
- **Sesgo de supervivencia residual:** la habilidad es una variable oculta y puede seguir contaminando el efecto de las materias de años altos.
- **Extrapolación a materias no habilitadas:** el modelo se entrenó solo con materias habilitadas y se usa con un perfil proyectado para el resto.
- **Modelo lineal:** la regresión logística no captura interacciones entre variables.
- **Interpretación del algoritmo:** el costo de Dijkstra no es la suma de todos los requisitos; el resultado principal es el orden de cursada. No se modelan cuatrimestres ni disponibilidad de la materia por cuatrimestre.
- **Tamaño de muestra:** algunas materias de años altos tienen pocos datos, lo que genera tasas extremas (por ejemplo, 1.0).
- **Rango de validez:** notas de 6 a 10, hasta 5 recursadas y 1 a 4 materias por cuatrimestre.
- **Persistencia:** el historial vive en el estado de la página y se pierde al recargar.

Extensiones

- Comparar con otros modelos (árbol de decisión, KNN) y con un baseline.
- Agrupar la sugerencia por cuatrimestre, recalculando las probabilidades a medida que se simula la aprobación.
- Selector de estudiantes de ejemplo y de materia objetivo.
- Mostrar el camino de mayor riesgo (cuello de botella) con nx.dag_longest_path.
- Reemplazar la dificultad subjetiva por tasas reales de aprobación.
- Guardar el historial con localStorage o en el backend.

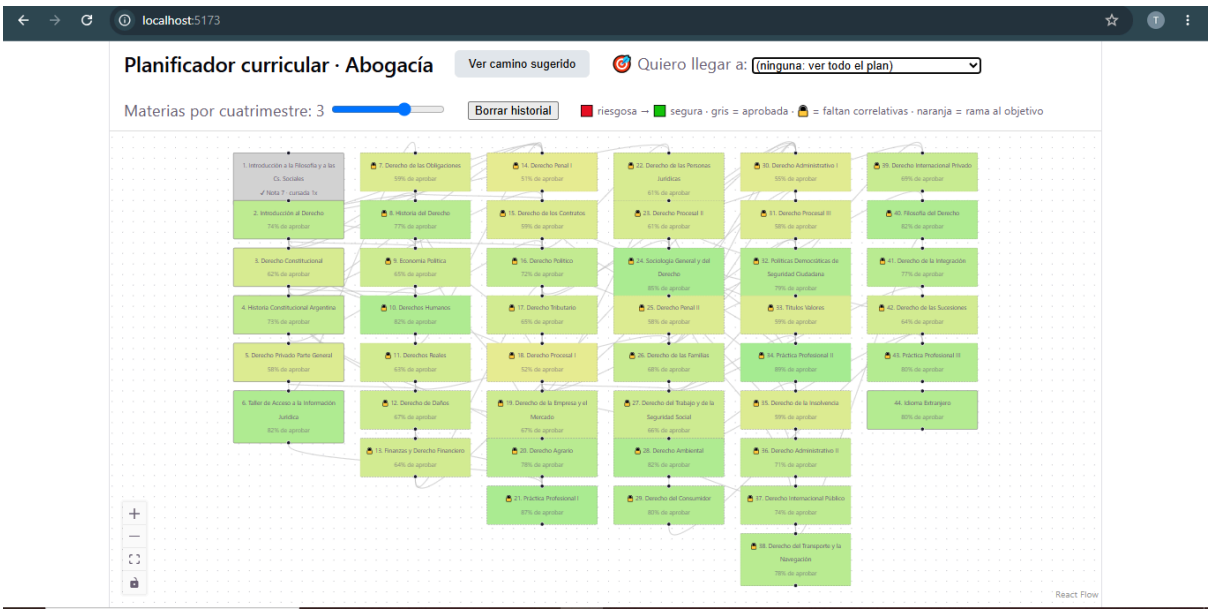
9. Instrucciones de instalación y ejecución

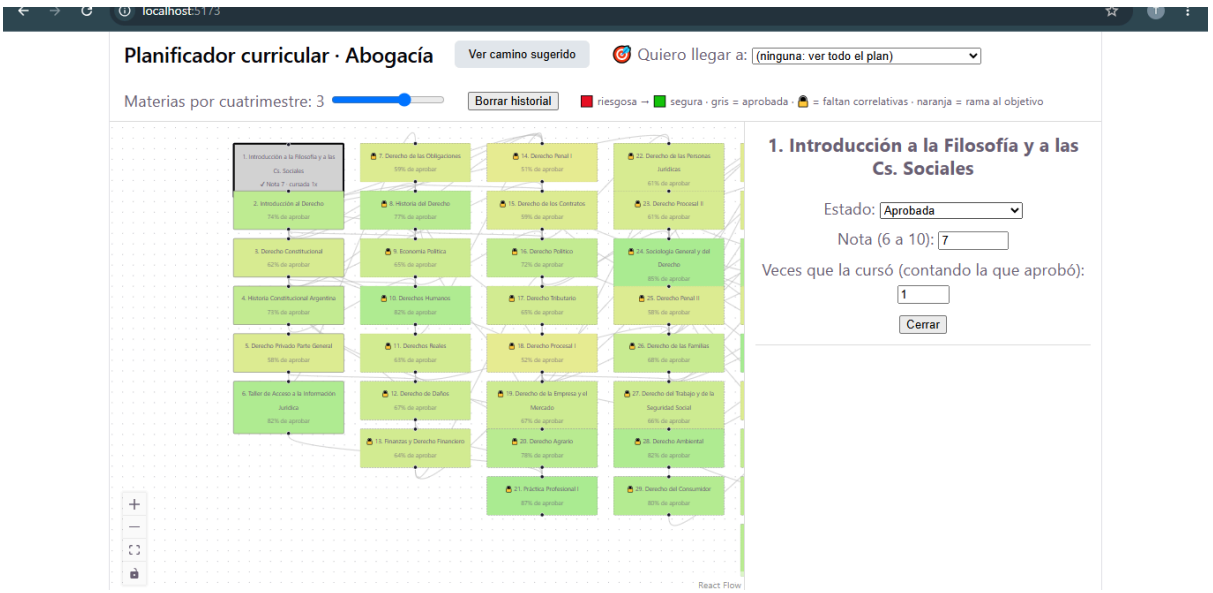
Desde la raíz del repositorio:

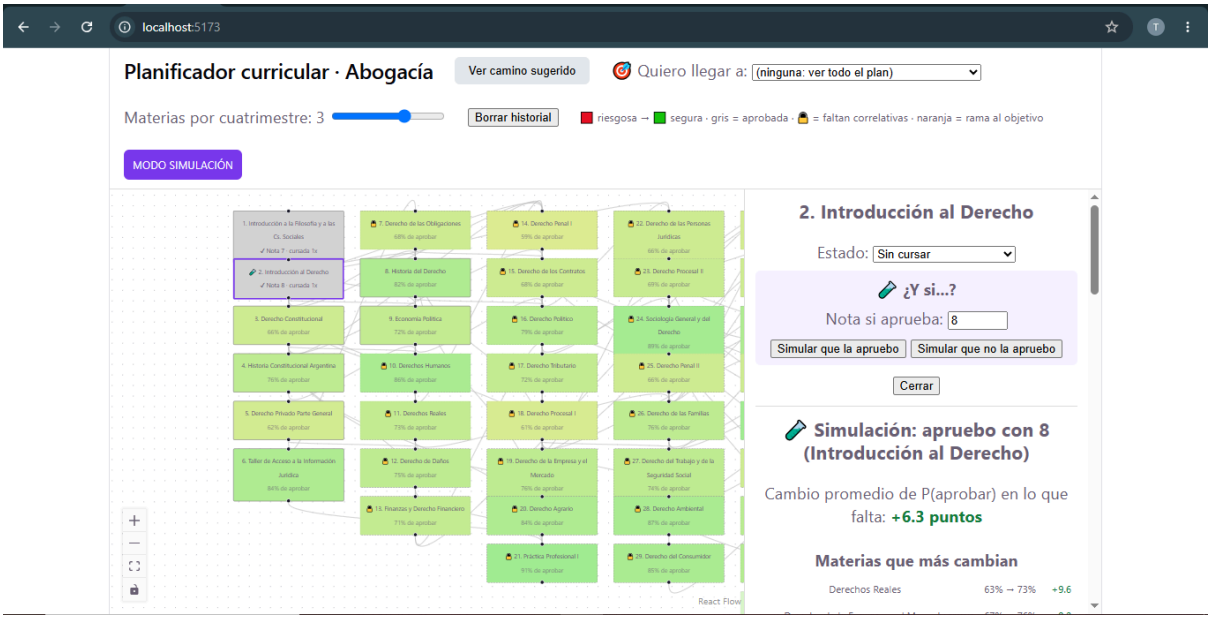
1. Instalar dependencias de Python: python -m pip install fastapi uvicorn pandas scikit-learn networkx joblib matplotlib.
2. Generar el dataset: python -m backend.data.generador.
3. Entrenar el modelo: python -m backend.prediccion.
4. Levantar la API: python -m uvicorn backend.api:app --reload (queda en http://localhost:8000).
5. En otra terminal: cd frontend, npm install, npm run dev (queda en http://localhost:5173).

Todos los comandos de Python se ejecutan desde la raíz del repositorio.}

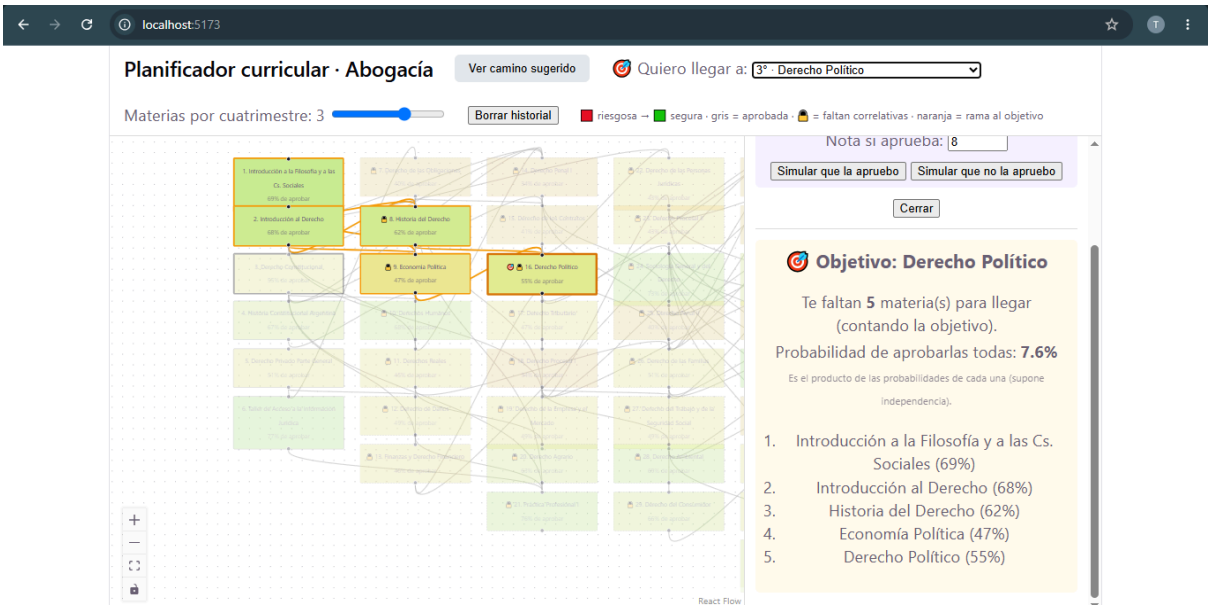
10. Capturas de pantalla del frontend






localhost:5173
Planificador curricular · Abogacía
Ver camino sugerido
Quiero llegar a: (ninguna: ver todo el plan)
Materias por cuatrimestre: 3
Borrar historial
riesgosa → segura · gris = aprobada · = faltan correlativas · naranja = rama al objetivo
MODO SIMULACIÓN
2. Introducción al Derecho
Estado: Sin cursar
¿Y si...?
Nota si aprueba: 8
Simular que la apruebo
Simular que no la apruebo
Cerrar
Simulación: apruebo con 8 (Introducción al Derecho)
Cambio promedio de P(aprobar) en lo que falta: +6.3 puntos
Materias que más cambian
Derechos Reales 63% → 73% +9.6
React Flow


localhost:5173
Planificador curricular · Abogacía
Ocultar camino sugerido
Quiero llegar a: (ninguna: ver todo el plan)
Materias por cuatrimestre: 3
Borrar historial
riesgosa → segura · gris = aprobada · = faltan correlativas · naranja = rama al objetivo
Orden de cursada que respeta las correlatividades y prioriza las materias de menor riesgo.
1. Taller de Acceso a la Información Jurídica (82%)
2. Idioma Extranjero (80%)
3. Introducción al Derecho (74%)
4. Historia del Derecho (77%)
5. Historia Constitucional Argentina (73%)
6. Economía Política (65%)
7. Derecho Constitucional (62%)
8. Derechos Humanos (82%)
9. Derecho Privado Parte General (58%)
10. Derecho Político (72%)
11. Derecho Penal I (51%)
12. Sociología General y del Derecho (85%)
13. Finanzas y Derecho Financiero (64%)
14. Derecho de Daños (67%)
15. Filosofía del Derecho (82%)
16. Derecho de las Obligaciones (59%)
React Flow


localhost:5173
Planificador curricular · Abogacía
Ver camino sugerido
Quiero llegar a: 3° · Derecho Político
Materias por cuatrimestre: 3
Borrar historial
riesgosa → segura · gris = aprobada · = faltan correlativas · naranja = rama al objetivo
Nota si aprueba: 8
Simular que la apruebo
Simular que no la apruebo
Cerrar
Objetivo: Derecho Político
Te faltan 5 materia(s) para llegar (contando la objetivo).
Probabilidad de aprobarlas todas: 7.6%
Es el producto de las probabilidades de cada una (supone independencia).
1. Introducción a la Filosofía y a las Cs. Sociales (69%)
2. Introducción al Derecho (68%)
3. Historia del Derecho (62%)
4. Economía Política (47%)
5. Derecho Político (55%)
React Flow